

Teste de Caixa Branca

Sistema de Pedidos



Atividade: Teste de Caixa Branca — Sistema de Pedidos

Aluno: Alice ♥

Turma: 3º ano B

Professores: Reenye e Robson

Data: 02/10/2026

1. Contextualização sobre Teste de Caixa Branca

O teste de caixa branca é uma técnica de teste de software que considera a estrutura interna do programa. Diferentemente de testes que analisam somente entradas e saídas, o teste de caixa branca permite verificar comandos, condições, decisões, caminhos de execução e alterações realizadas nas variáveis.

Nesta atividade foi analisado um sistema de pedidos desenvolvido em HTML, CSS e JavaScript. O sistema permite selecionar um produto, informar uma quantidade, utilizar cupons de desconto e escolher uma modalidade de frete.

Durante o processamento, o programa:

1. lê os dados informados;
2. valida a quantidade;
3. verifica o estoque;
4. calcula o subtotal;
5. calcula o desconto;
6. calcula o frete;

7. calcula o total;
8. aplica possíveis descontos adicionais;
9. classifica o pedido;
10. apresenta o resultado.

A análise foi realizada utilizando principalmente **cobertura de decisões, análise de caminhos, valores-limite, condições e rastreamento de variáveis**.

2. Análise das Estruturas de Decisão

O código possui as seguintes estruturas relevantes.

Validação da quantidade

```
if (qtd < 0) {  
    resultado.innerHTML = "<p>Quantidade inválida.</p>";  
    return;  
}
```

A intenção é impedir quantidades inválidas. Como a quantidade mínima permitida é 1, o valor 0 também deve ser rejeitado.

Verificação do estoque

```
if (qtd >= estoque[produtoSelecionado]) {  
    resultado.innerHTML = "<p>Quantidade indisponível em estoque.</p>";  
    return;  
}
```

A quantidade disponível em estoque deve poder ser comprada. Portanto, a comparação deve permitir uma quantidade exatamente igual ao estoque.

Cupom SENAI10

```
if (codigo === "SENAI10") {  
    return subtotal * 0.10;  
}
```

O cupom concede 10% de desconto.

Cupom SENAI20

```
if (codigo === "SENAI20" && subtotal >= 1000) {  
    return subtotal * 0.20;  
}
```

O cupom concede 20% para pedidos de pelo menos R\$ 1.000,00.

Frete

```
if (tipo === "retirada") {  
    return 0;  
}  
  
if (tipo === "expresso") {  
    return 60;  
}  
  
if (subtotal >= 500) {  
    return 0;  
}  
  
return 30;
```

A retirada é gratuita, o frete expresso custa R\$ 60,00 e o frete normal é gratuito para subtotal a partir de R\$ 500,00.

Desconto por quantidade

```
if (qtd > 5) {  
    total = total - subtotal * 0.05;  
}
```

Considerando que a regra seja "5 unidades ou mais", o limite deveria incluir a quantidade 5.

Desconto de alto valor

```
if (total > 3000) {  
    total = total * 0.95;  
}
```

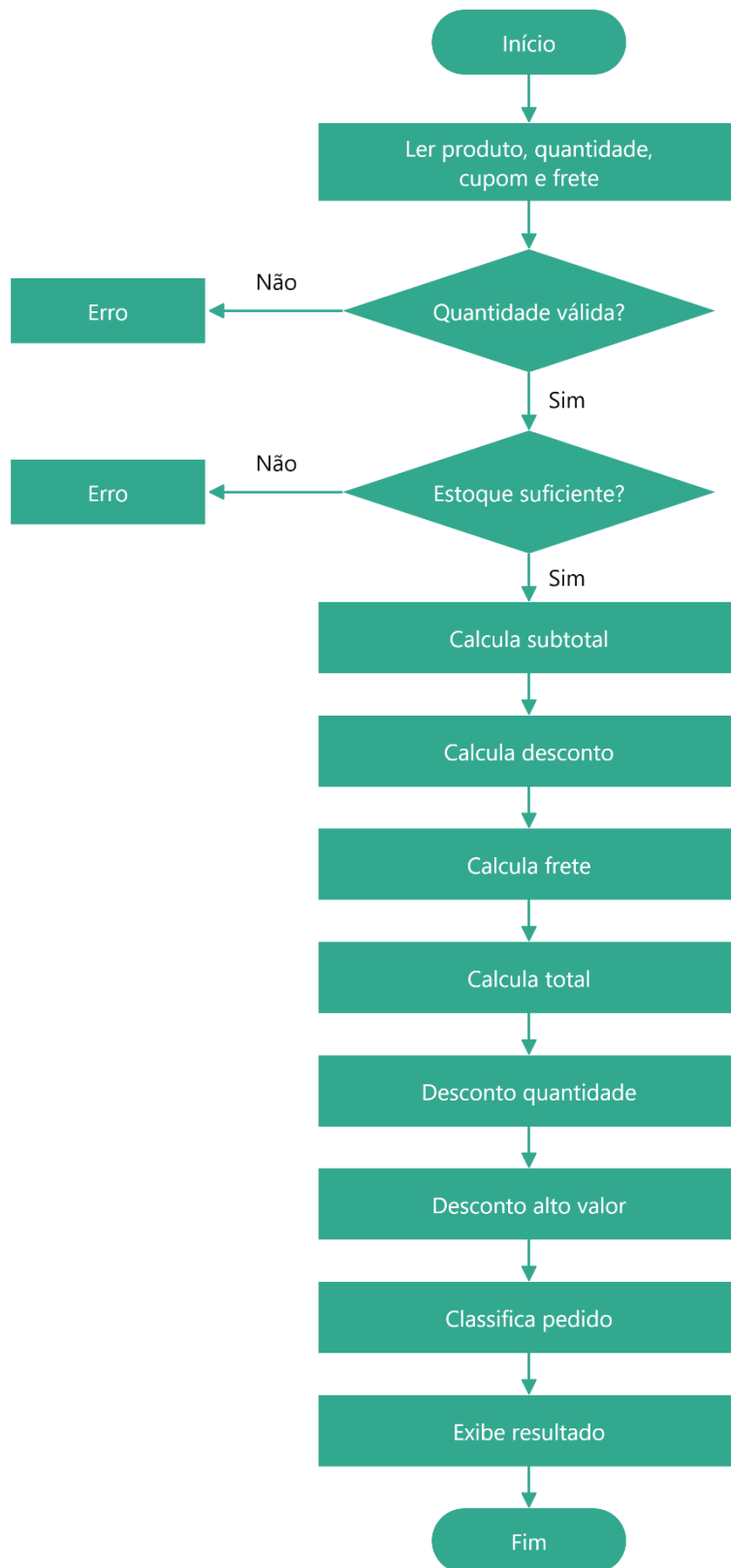
Considerando que pedidos de R\$ 3.000,00 ou mais recebem o desconto adicional, o limite deveria incluir exatamente R\$ 3.000,00.

Classificação do pedido

```
if (total <= 0) {  
    mensagem = "Valor do pedido inválido.";   
} else if (total >= 3000) {  
    mensagem = "Pedido de alto valor.";   
}
```

Aqui existe uma interação importante com o desconto de alto valor: depois que o desconto é aplicado, o total pode deixar de ser R\$ 3.000,00 ou mais.

3. Fluxo geral



4. Casos de teste

Uma matriz inicial de testes pode ser:

Teste	Produto	Qtd.	Cupom	Frete	Objetivo
CT01	Mouse	0	-	Normal	Testar limite inferior
CT02	Notebook	5	-	Normal	Testar limite do estoque
CT03	Mouse	5	-	Normal	Testar desconto por quantidade
CT04	Notebook	1	-	Normal	Testar limite de R\$ 3.000
CT05	Notebook	1	SENAI20	Normal	Testar limite do cupom
CT06	Teclado	4	SENAI10	Retirada	Testar combinação de caminhos

Os seis erros abaixo são os pontos de falha encontrados.

ERRO 1

Nível: Fácil

Técnica usada: análise de valores-limite

Trecho do código:

```
if (qtd < 0) {  
    resultado.innerHTML = "<p>Quantidade inválida.</p>";  
    return;  
}
```

Comportamento esperado: quantidade 0 deve ser considerada inválida.

Dados do teste: mouse, quantidade 0, sem cupom, frete normal.

Caminho percorrido: D6 (0 < 0) é falso. O programa continua e calcula o pedido com quantidade 0.

Resultado esperado: "Quantidade inválida."

Resultado obtido: o sistema continua o processamento e apresenta total R\$ 0,00.

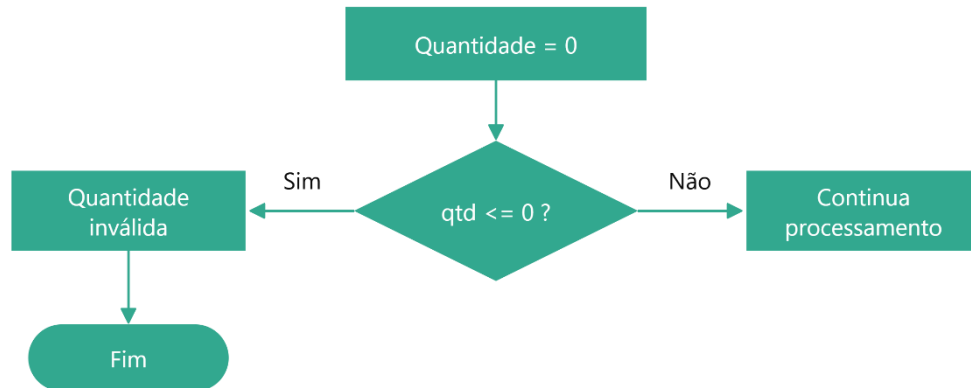
Erro identificado: o < não considera o valor 0 inválido. O correto é <=.

Correção realizada:

```
if (qtd <= 0) {
```

Resultado após a correção: com quantidade 0, o sistema mostra "Quantidade inválida."

Fluxograma:



ERRO 2

Nível: Fácil

Técnica usada: análise de valores-limite

Trecho do código:

```
if (qtd >= estoque[produtoSelecionado]) {  
    resultado.innerHTML = "<p>Quantidade indisponível em estoque.</p>";  
    return;  
}
```

Comportamento esperado: se existem 10 teclados em estoque, dá para comprar os 10.

Dados do teste: teclado, quantidade 10, sem cupom, retirada.

Caminho percorrido: D6 ($10 < 0$) é falso. Em D7, $10 \geq 10$ é verdadeiro, então o programa mostra "indisponível" e encerra com return.

Resultado esperado: pedido aceito, com subtotal R\$ 1.500,00, desconto R\$ 75,00 e total R\$ 1.425,00.

Resultado obtido: "Quantidade indisponível em estoque."

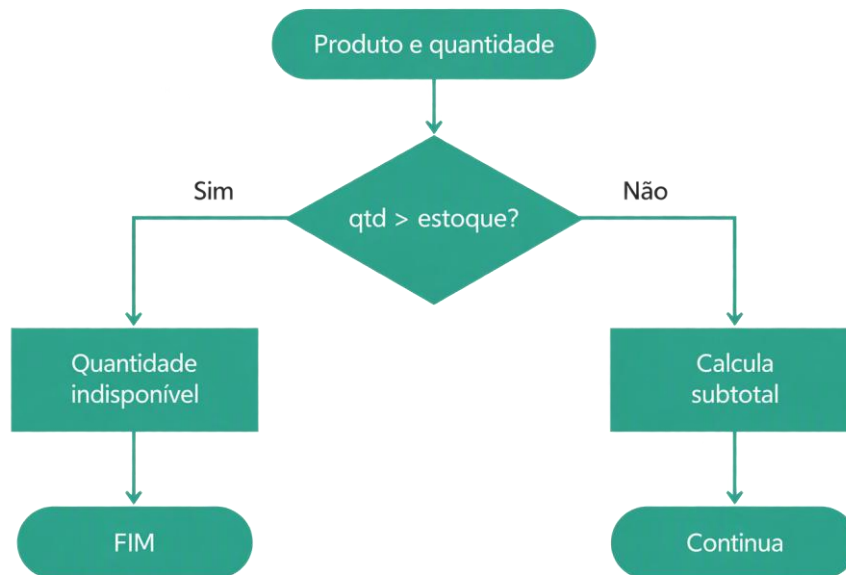
Erro identificado: o $\geq$ bloqueia o pedido que utiliza todo o estoque. O correto é $>$.

Correção realizada:

```
if (qtd > estoque[produtoSelecionado]) {
```

Resultado após a correção: o pedido é aceito, com total R\$ 1.425,00. Com quantidade 11, continua retornando "Quantidade indisponível em estoque."

Fluxograma:



ERRO 3

Nível: Médio

Técnica usada: análise de valores-limite

Trecho do código:

```
if (qtd > 5) {  
    total = total - subtotal * 0.05;  
}
```

Comportamento esperado: pedidos com 5 unidades ou mais recebem 5% de desconto.

Dados do teste: mouse, quantidade 5, sem cupom, frete normal.

Caminho percorrido: D6 e D7 são falsos. O subtotal é R\$ 400,00 e o total com frete é R\$ 430,00. Em D10, $5 > 5$ é falso, então o desconto não é aplicado.

Resultado esperado: desconto de R\$ 20,00 e total R\$ 410,00.

Resultado obtido: desconto não aplicado e total R\$ 430,00.

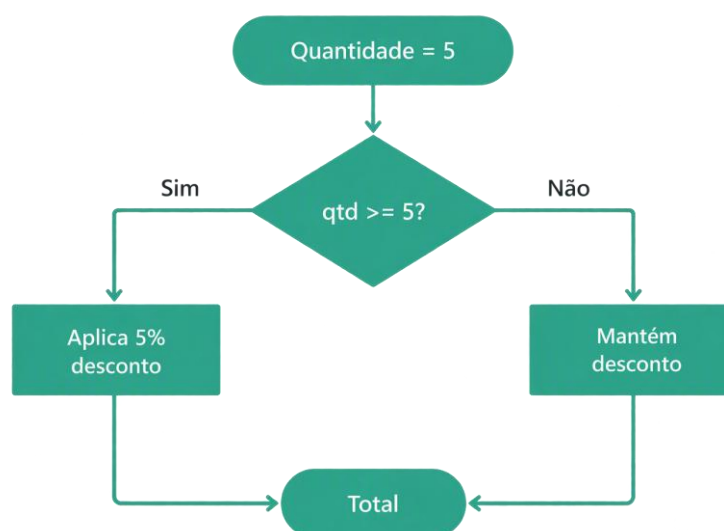
Erro identificado: o $>$ exclui a quantidade 5. O correto é $\geq$.

Correção realizada:

```
if (qtd >= 5) {
```

Resultado após a correção: com 5 unidades, o sistema aplica R\$ 20,00 de desconto e o total fica R\$ 410,00.

Fluxograma:



ERRO 4

Nível: Médio

Técnica usada: análise de valores-limite

Trecho do código:

```
if (total > 3000) {  
    total = total * 0.95;
```

```
}
```

Comportamento esperado: pedidos de R\$ 3.000,00 ou mais recebem 5% de desconto.

Dados do teste: notebook, quantidade 1, sem cupom, retirada.

Caminho percorrido: subtotal = R\$ 3.000,00 e frete = R\$ 0,00. Em D11, $3000 > 3000$ é falso, então o desconto não é aplicado.

Resultado esperado: desconto de R\$ 150,00 e total R\$ 2.850,00.

Resultado obtido: total R\$ 3.000,00, sem o desconto.

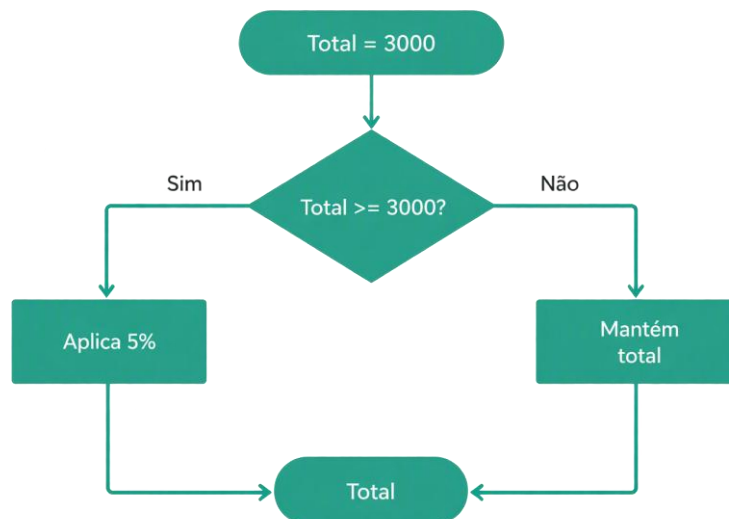
Erro identificado: o $>$ não inclui exatamente R\$ 3.000,00. O correto é $\geq$.

Correção realizada:

```
if (total >= 3000) {
```

Resultado após a correção: o sistema aplica 5% e apresenta total de R\$ 2.850,00.

Fluxograma:



ERRO 5

Nível: Difícil

Técnica usada: rastreamento de variáveis

Trecho do código:

```
if (total > 3000) {  
    total = total * 0.95;  
}  
  
...  
  
else if (total >= 3000) {  
    mensagem = "Pedido de alto valor";  
}
```

Comportamento esperado: a classificação de alto valor deve considerar o valor do pedido antes do desconto de alto valor.

Dados do teste: notebook, quantidade 1, sem cupom, retirada.

Caminho percorrido: o total começa em R\$ 3.000,00. Após o desconto de 5%, passa para R\$ 2.850,00. Na classificação, $2.850 \geq 3.000$ é falso.

Resultado esperado: "Pedido de alto valor."

Resultado obtido: "Pedido calculado com sucesso."

Erro identificado: a variável total é alterada antes de ser utilizada na classificação.

Correção realizada:

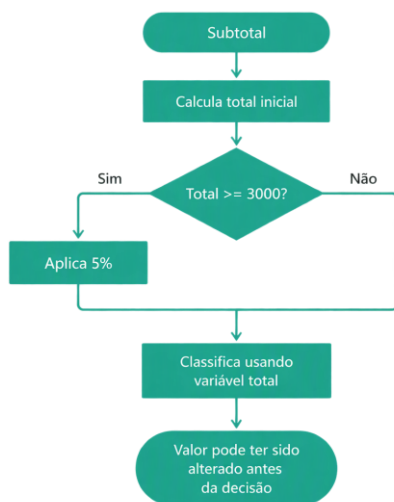
```
const totalAntesDescontoAltoValor = total;
```

E na classificação:

```
else if (totalAntesDescontoAltoValor >= 3000) {
```

Resultado após a correção: o total fica R\$ 2.850,00 e a mensagem é "Pedido de alto valor."

Fluxograma:



ERRO 6

Nível: Difícil

Técnica usada: análise de condições compostas

Trecho do código:

```

if (codigo === "SENAI20" && subtotal >= 1000) {
    return subtotal * 0.20;
}
  
```

Comportamento esperado: o cupom SENAI20 deve dar 20% de desconto quando o subtotal for pelo menos R\$ 1.000,00.

Dados do teste: teclado, quantidade 7, cupom SENAI20, retirada.

Caminho percorrido: subtotal = R\$ 1.050,00. A primeira condição é verdadeira (SENAI20) e a segunda também ($1050 \geq 1000$). O desconto de 20% é aplicado.

Resultado esperado: desconto de R\$ 210,00 e total R\$ 840,00, considerando também o desconto de quantidade.

Resultado obtido: desconto de R\$ 210,00 e total R\$ 787,50.

Erro identificado: o desconto de quantidade é aplicado sobre o subtotal **depois** do desconto do cupom, causando uma diferença em relação à regra esperada de aplicar 5% sobre o subtotal original.

Correção realizada:

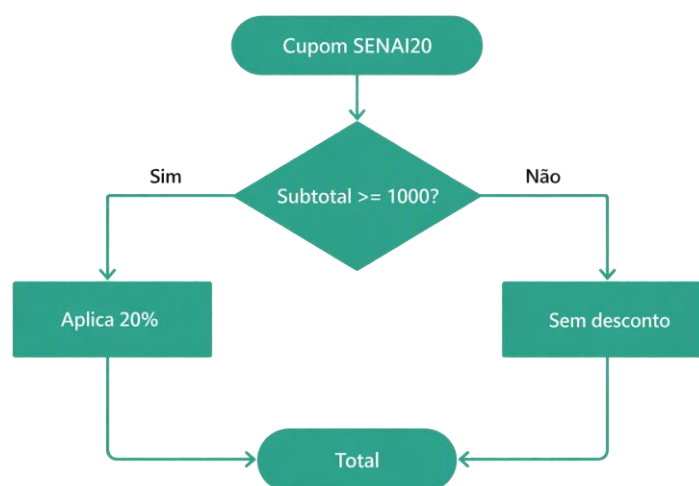
```

if (qtd >= 5) {
    total = total - subtotal * 0.05;
}

```

Resultado após a correção: o desconto de quantidade passa a ser calculado sobre o subtotal original, mantendo o cálculo esperado.

Fluxograma:



6. Testes após as correções

Teste	Entrada	Resultado esperado	Resultado após correção	Situação
CT01	Mouse, qtd. 0	Quantidade inválida	Quantidade inválida	OK
CT02	Notebook, qtd. 5	Processar pedido	Pedido processado	OK
CT03	Mouse, qtd. 5	Aplicar 5%	R\$ 410,00	OK
CT04	Notebook, qtd. 1, retirada	Aplicar regra de alto valor	R\$ 2.850,00	OK
CT05	Mouse, qtd. 10, SENAI20	Sem desconto abaixo de R\$ 1.000	R\$ 0 de cupom	OK

7. Comparação antes e depois

Situação	Antes da correção	Depois da correção
Quantidade 0	Processava	Rejeitada
Compra de todo o estoque	Rejeitava	Permitida
5 unidades	Não recebia desconto	Recebe 5%
Total R\$ 3.000	Não recebia desconto de alto valor	Recebe 5%
Classificação após alteração do total	Dependia de total já modificado	Pode utilizar o valor original

8. Conclusão

A análise demonstrou que o teste de caixa branca permite identificar problemas que podem não ser percebidos simplesmente observando o resultado final de alguns testes.

Os principais problemas encontrados estão relacionados a **valores-limite e condições de decisão**, principalmente nos operadores $<$, $>$, $>=$ e na alteração de variáveis antes de decisões posteriores.

A análise dos caminhos permitiu acompanhar a execução desde a entrada dos dados até a apresentação do resultado, identificando em qual decisão o comportamento real deixou de corresponder ao comportamento esperado.

Também foi possível observar a importância do rastreamento de variáveis. A variável total, por exemplo, é modificada durante o processamento e posteriormente utilizada para outra decisão. Dessa forma, a ordem das operações interfere diretamente no resultado.

Os testes realizados após as correções demonstram que as alterações modificaram os caminhos problemáticos e fizeram o programa atender às regras de negócio consideradas na análise.